

Task 1 — GUI Checklist: Design, Execution, and Bug Report

Student ID: 23127296 **SUT:** EShop (eshop-clone) — frontend-web. **Scope selected (§5):** Home/Search, Login, Register, Forgot Password (2-step), Profile + Order History, Product Detail, Cart, Checkout + Coupon — eight frontend-web screens/flows, chosen so every failing item is reproducible and screenshot-able solo, without an admin account or a second device.

1. Process

Step 1 — AI-generated baseline (39 items)

An initial checklist pass was produced by prompting the AI with the four required interface aspects (IA-01 General UI, IA-02 Forms, IA-03 Navigation, IA-04 Feedback/state) and the list of in-scope screens, asking it to generate concrete, screen-specific checklist items rather than one generic “find usability problems” prompt. This produced 39 items covering standard heuristics: visual consistency, color semantics, hover/focus states, empty-state handling, confirmation dialogs, field hints, loading states, etc. — see rows marked `Origin=AI` in `../01_gui_checklist/GUI_Checklist.csv`.

Step 2 — Critical human review (3 added items)

The AI-generated set was reviewed by actually running the corresponding flow on the live app, and 3 additional items were added where the baseline pass had no way to know something was wrong without executing the flow and comparing what appeared on screen against another screen or against `api_specification.md`:

Item	Why the AI baseline missed it
OTP field label vs. actual code length (Forgot Password)	Catching this requires running the flow and comparing the code that comes back on screen against the field's own label — a pure UI-copy read-through has no code to compare against until the flow is executed.
Search box renders user input as HTML instead of literal text (Home)	Confirming this requires typing an HTML/script payload into the search box and watching whether it executes — a visual-only pass typing normal search terms never triggers it.
Product-not-found state shows a debug string, not a user message (Product Detail)	Only reached by manually navigating to a non-existent product ID — a walkthrough that only opens products via Home's own links never lands on this state.

This matches the assignment's own framing: the AI baseline is a fast generator of standard-heuristic coverage; the code/flow-grounded review is what catches semantic mismatches (UI copy vs. actual behavior) and states that only appear under a specific, deliberately-chosen input.

Step 3 — Execution

All 42 items were executed against the running app (`npm run dev` on frontend-web, backend on `http://localhost:3000`) by clicking through each screen, filling forms with valid and deliberately edge-case inputs, and cross-checking on-screen values (OTP length, checkout total, cart contents) against what the app itself displayed elsewhere or against `api_specification.md`. Results, Pass/Fail, and per-failure notes are recorded in `../01_gui_checklist/GUI_Checklist.csv`.

Step 4 — Bug logging

All 10 failed items were written up as bug reports in `../02_bug_reports/Bug_Report.md` (severity, repro steps, expected/actual) and filed as real GitHub Issues on `KidCute1412/eshop-sut`, each with a screenshot attached. Issue links are recorded in both the Bug Report's index table and the checklist's `Bug_ID` column.

2. Results

See `../01_gui_checklist/Test_Summary.md` for the full numeric breakdown:

- 42 items designed, 42 executed — **32 Passed / 10 Failed**.
- Coverage by IA: IA-01 General UI (10 items, 1 failed), IA-02 Forms (12 items, 5 failed), IA-03 Navigation (7 items, 1 failed), IA-04 Feedback/state (13 items, 3 failed).
- 2 of the 10 failures are **Critical** security issues (plaintext password field — BUG-11; reflected XSS via search — BUG-19); 5 are **Major** (BUG-13, BUG-14, BUG-15, BUG-24, BUG-25); 3 are **Minor**/cosmetic (BUG-01, BUG-34, BUG-37).
- All 10 bugs are filed as GitHub Issues with screenshots — see `../02_bug_reports/Bug_Report.md` for the full list and issue links.